

# Simple Documentación del Pipeline

TFG<sup>1\*</sup>

<sup>1\*</sup>FPUNA

...@fpuna.edu.py

July 29, 2025

## Abstract

En el presente proyecto se aborda el problema de la predicción del riesgo de incendios forestales en el Departamento de Cordillera, Paraguay, haciendo uso de un pipeline automatizado de procesamiento de datos geoespaciales y un modelo avanzado de Machine Learning. El marco temporal del estudio se define en función del histórico de datos de focos de calor provistos por el sistema FIRMS de la NASA.

Se integran múltiples variables predictoras correspondientes a categorías topográficas (elevación, pendiente, orientación), meteorológicas (temperatura, precipitación, viento, humedad), de vegetación (NDVI), de cobertura del suelo y de proximidad a infraestructuras humanas (distancia a vías y ciudades). Para la generación de la muestra, se utilizan los puntos de focos de calor como observaciones positivas y se implementa una estrategia de muestreo de fondo espacio-temporal para generar observaciones negativas (no-incendio) de alta calidad, evitando el sesgo de un muestreo puramente aleatorio.

Se genera una muestra final balanceada que se utiliza para entrenar y evaluar el modelo. Críticamente, para evitar una evaluación sesgada por la autocorrelación espacial, se emplea una validación por bloques espaciales en lugar de una partición aleatoria. El modelo central implementado es un **StackingClassifier**, un método de ensamble que combina las predicciones de varios estimadores base (Random Forest, Support Vector Machine y k-Nearest Neighbours) a través de un meta-modelo de Regresión Logística para mejorar la robustez y precisión predictiva.

Se evalúa el rendimiento del modelo sobre un conjunto de prueba geográficamente aislado, utilizando métricas clave como Accuracy, ROC AUC, la matriz de confusión y el reporte de clasificación. El modelo de ensamble demuestra un rendimiento prometedor en la distinción entre condiciones de incendio y no-incendio. Finalmente, el pipeline culmina con la aplicación del modelo entrenado para generar un mapa de riesgo de alta resolución para una fecha específica, proporcionando una herramienta visual e interpretable para la gestión y mitigación de incendios en la región.

## 1. Librerías

### 1.1. Manipulación y Análisis de Datos Fundamentales

- **pandas:** (import pandas as pd) librería principal para la manipulación y el análisis de datos estructurados. Se utilizó para leer los archivos .csv que contienen los focos de calor (FIRMS) y los datos climáticos (temperatura, precipitación, etc.). Permite combinar estas diferentes fuentes de datos, manejar las series temporales con la columna date, y organizar toda la información en un formato de DataFrame, que es la estructura de datos central a lo largo de todo el proyecto.
- **numpy:** (import numpy as np) paquete fundamental para la computación numérica. Se empleó para fijar las semillas aleatorias (np.random.seed) garantizando la reproducibilidad de los resultados. También se usó para generar los puntos aleatorios de la muestra negativa y para crear la rejilla (meshgrid) sobre la que se genera el mapa de riesgo fi-

nal. Muchas otras librerías, como pandas y scikit-learn, dependen de numpy para sus operaciones internas.

### 1.2. Manejo de Datos Geoespaciales (Vectoriales y Raster)

- **geopandas:** (import geopandas as gpd) extiende pandas para permitir el trabajo con datos geoespaciales vectoriales. Utilizado para leer los archivos de forma (.shp) del departamento, distritos, ciudades y vías. Permite convertir las coordenadas de los incendios en objetos geométricos (**points\_from\_xy**), filtrar los puntos que caen dentro del área de estudio (**.within**), y calcular distancias a elementos geográficos como vías y ciudades (**.distance**). También se usó para visualizar las capas vectoriales en los mapas.
- **shapely.geometry.Point:** (from shapely.geometry import Point) Proporciona los objetos geométricos que geopandas utiliza. Se usó directamente para crear objetos Point

a partir de coordenadas aleatorias durante la generación de las muestras negativas, antes de convertirlas a un `GeoDataFrame`.

- **rasterio:** (`import rasterio`) Es la librería principal para leer, escribir y manipular datos geoespaciales en formato raster (como archivos `.tif`). Se utilizó para abrir y leer los datos del Modelo Digital de Elevación (DEM) y los mapas de cobertura del suelo. La función `rasterio.features.shapes` fue clave para vectorizar las clases de cobertura inflamables, convirtiendo píxeles del raster en polígonos que definen las "zonas inflamables".
- **rioxarray:** Actúa como un puente entre `rasterio` y `xarray`, facilitando operaciones complejas en datos raster. Se usó para abrir los archivos raster (`.tif`) de DEM, Cobertura y NDVI como `DataArrays` de `xarray`. Su función más importante en el script fue la reproyección (`.rio.reproject()`) del DEM a un sistema de coordenadas métrico (UTM), un paso indispensable para calcular correctamente la pendiente y la orientación.
- **xarray:** (`xr`) Permite trabajar con arreglos N-dimensionales etiquetados, ideal para los rasters. Se utilizó para crear `DataArrays` a partir de las coordenadas de los puntos. Su método de selección basado en etiquetas (`.sel(method="nearest")`) fue fundamental para extraer de manera eficiente los valores de los rasters (elevación, pendiente, NDVI, etc.) correspondientes a la ubicación de cada punto de la muestra.
- **xrspatial:** (`xrs`) es una extensión de `xarray` para análisis espaciales basados en rasters. Se utilizó para calcular las variables topográficas de pendiente (`xrs.slope()`) y orientación (`xrs.aspect()`) a partir del DEM reproyectado.

### 1.3. Modelado Predictivo y Evaluación (Machine Learning)

- **scikit-learn (sklearn):** librería utilizado para realizar tareas de aprendizaje automático supervisado y no supervisado.
  - **-preprocessing:** Se usó `StandardScaler` para normalizar las variables numéricas (como elevación o temperatura) y `OneHotEncoder` para convertir las variables categóricas (como el tipo de cobertura) en un formato numérico que el modelo pueda procesar.
  - **-compose.ColumnTransformer:** Permitió aplicar los diferentes pasos

de preprocesamiento (escalado y codificación) a las columnas correspondientes de forma organizada.

- **-pipeline.Pipeline:** Se utilizó para encadenar el preprocesador y el modelo clasificador en un único objeto. Esto asegura que el mismo procesamiento se aplique de forma consistente a los datos de entrenamiento, prueba y predicción, evitando errores y fugas de datos.
- **-ensemble.RandomForestClassifier, svm.LinearSVC, neighbors.KNeighborsClassifier:** Estos fueron los tres modelos base utilizados en el ensamble.
- **-ensemble.StackingClassifier:** Fue el núcleo del modelado, permitiendo combinar las predicciones de los tres modelos base para obtener un resultado final más preciso y robusto.
- **-linear\_model.LogisticRegression:** Se usó como el meta-modelo dentro del `StackingClassifier`, encargado de aprender a combinar las predicciones de los modelos base.
- **-calibration.CalibratedClassifierCV:** Se usó para envolver al `LinearSVC`, ya que este último no produce probabilidades de forma nativa. Este calibrador le permite generar las estimaciones de probabilidad necesarias para el `StackingClassifier` y el cálculo del AUC-ROC.
- **-metrics:** Se usaron funciones como `accuracy_score`, `confusion_matrix`, `classification_report` y `roc_auc_score` para realizar una evaluación exhaustiva del rendimiento del modelo final.

### 1.4. Explicabilidad e Interpretación del Modelo

- **shap:** Se utilizó para explicar las predicciones del modelo `RandomForest`. Se creó un `TreeExplainer` para calcular los valores SHAP, que miden el impacto de cada variable en la predicción. Con ellos se generaron los gráficos de resumen (`summary_plot`) que visualizan qué variables son las más importantes y cómo influyen en el riesgo de incendio.

### 1.5. Visualización de Datos y Resultados

- **matplotlib.pyplot:** Es la librería fundamental para crear todo tipo de gráficos y visualizaciones. Se utilizó para crear la estructura de todas las figuras, configurar títulos, ejes, guardar los gráficos en archivos (`.png`) y mostrarlos en pantalla.

- **seaborn:** Construida sobre matplotlib, facilita la creación de gráficos estadísticos más complejos y estéticamente agradables. Se usó para generar los diagramas de caja (box-plot) que comparan las distribuciones de las variables entre los casos de incendio y no-incendio, y para crear el mapa de calor (heatmap) de la matriz de confusión.
- **scipy.interpolate.griddata:** función de la librería científica SciPy esencial para crear el mapa de riesgo final. Interpoló las probabilidades de incendio predichas en puntos dispersos para crear una superficie continua y suave (un heatmap), que es mucho más fácil de interpretar visualmente.
- **contextily:** utilizadp para agregar un mapa de fondo a las visualizaciones espaciales (cx.add.basemap), lo que proporciona un contexto geográfico invaluable para entender dónde se localizan los puntos de la muestra y las zonas de alto riesgo.
- **matplotlib.patches y matplotlib.path:** Módulos para crear formas y trazados complejos. Se utilizaron para crear una máscara de recorte a partir del polígono del departamento. Esto permitió que el mapa de calor interpolado se mostrara únicamente dentro de los límites del área de estudio.

## 1.6. Librerías Estándar de Python

- **os:** Proporciona funciones para interactuar con el sistema operativo. Para establecer una variable de entorno (PYTHONHASHSEED) con el fin de mejorar la reproducibilidad.
- **datetime:** para trabajar con fechas y horas y así crear un nombre de carpeta único (TIMESTAMP) donde guardar todos los resultados de una ejecución.
- **pathlib.Path:** Se utilizó de forma extensiva para definir todas las rutas a los datos de entrada y a las carpetas de salida.
- **shutil:** para copiar los resultados más importantes (mapas, reportes) desde la carpeta de salida con timestamp a una carpeta fija (frontend/output), preparándolos para ser consumidos por una aplicación web o dashboard.
- **io:** para capturar la salida de texto del resumen del DataFrame (.info()) como una cadena de texto, para luego guardarla en un archivo.

## 2. Funciones Auxiliares

### 2.1. Función load\_firms\_data

- **Propósito:** Cargar, filtrar y formatear los datos de focos de calor del sistema FIRMS (Fire Information for Resource Management System).
- **Parámetros:**
  - **firms\_folder:** Ruta a la carpeta que contiene los archivos .csv con los datos de focos de calor.
  - **study\_area\_geom:** Objeto de geometría poligonal que define los límites del área de estudio (el departamento de Cordillera).
  - **crs:** Sistema de Coordenadas de Referencia (CRS) del área de estudio, para asegurar la coherencia espacial.
- **Proceso:**
  1. Localiza y lee todos los archivos .csv dentro de la carpeta especificada y los combina en un único DataFrame de pandas.
  2. Convierte la columna de fecha (acq\_date) a formato datetime.
  3. Transforma el DataFrame en un GeoDataFrame de geopandas, usando las columnas de latitud y longitud.
  4. Filtra espacialmente: descarta todos los puntos fuera del polígono del área de estudio.
  5. Añade una columna fire con el valor 1, etiquetando estos puntos como positivos (presencia de incendio).
  6. Devuelve solo las columnas esenciales: fecha, geometría y etiqueta.
- **Retorno:** Un GeoDataFrame que contiene exclusivamente los focos de calor ocurridos dentro del área de estudio, listo para ser usado como muestra positiva.

### 2.2. Función load\_climate\_data

- **Propósito:** Cargar y unificar los diferentes archivos de datos climáticos en una única serie temporal diaria.
- **Parámetros:**
  - **temp\_path, precip\_path, wind\_path, humidity\_path:** Rutas a los archivos .csv de temperatura, precipitación, viento y humedad.

- **Proceso:**

1. Carga cada archivo de forma independiente, selecciona y renombra columnas.
2. Combina los `DataFrames` de temperatura, precipitación y viento.
3. Aplica `resample` para convertir a escala diaria: media para temperatura y viento, suma para precipitación.
4. Fusiona el `DataFrame` diario con los datos de humedad.
5. Rellena valores faltantes usando `ffill().bfill()`.

- **Retorno:** Un `DataFrame` indexado por fecha con datos climáticos diarios completos para las cuatro variables.

### 2.3. Función `asignar_variables`

- **Propósito:** Asignar variables predictoras a un conjunto de puntos a partir de fuentes de datos raster y vectoriales.

- **Parámetros:**

- `df_puntos`: El `GeoDataFrame` de puntos (incendio o no).
- `dem_path`, `cobertura_folder`, `ndvi_mensual_folder`: Rutas a los datos raster.
- `weather_df`: `DataFrame` de clima diario desde `load_climate_data`.
- `vias_shp_path`, `ciudades_shp_path`: Archivos de forma para el cálculo de distancias.

- **Proceso:**

1. **Topografía:**

- Extrae la elevación desde el DEM.
- Reproyecta el DEM a CRS métrico para calcular pendiente y orientación (usando `xrspatial`).
- Convierte la orientación a categorías cardinales.

2. **Cobertura del suelo:**

- Itera por año y extrae la clase de cobertura correspondiente para cada punto.

3. **Vegetación (NDVI):**

- Para cada punto, extrae el valor del raster NDVI mensual correspondiente al mes y año.

4. **Clima:**

- Asigna los valores climáticos del día correspondiente usando `merge_asof`.

5. **Proximidad:**

- Reproyecta a CRS métrico.
- Calcula distancia a vías principales (`dist_vias`) y a la ciudad más cercana (`dist_ciudades`).

- **Retorno:** El mismo `GeoDataFrame`, enriquecido con todas las variables predictoras necesarias para el modelado.

### 3. Generación de la muestra y asingación de datos

Esta fase del script se encarga de construir el conjunto de datos que servirá de base para todo el análisis. El proceso se divide en **tres pasos principales**: la preparación del área de estudio y las muestras positivas, la generación de muestras negativas y, finalmente, la consolidación y enriquecimiento del conjunto de datos.

#### 3.1. Preparación del Área de Estudio y Muestras Positivas (Incendios)

1. **Carga del Área de Estudio:** El proceso comienza cargando el archivo de forma (`.shp`) del departamento de Cordillera. La geometría y el Sistema de Coordenadas de Referencia (CRS) de este polígono (`area_estudio_gdf`) se convierten en la referencia espacial para todo el análisis, asegurando que todos los datos subsiguientes estén geográficamente alineados.

2. **Identificación de Zonas Inflamables:**

Se implementa una optimización crucial para mejorar la calidad de la muestra. Utilizando un mapa de cobertura del suelo como referencia, se identifican las clases que se consideran “inflamables” (por ejemplo, pastizales, bosques). Estos píxeles se convierten en un único polígono (`zonas_inflamables_poligono`). Este polígono actúa como una máscara, asegurando que el modelo se enfoque únicamente en las áreas donde un incendio es plausible.

3. **Carga de Puntos Positivos:**

Se invoca la función `load_firms_data` para cargar los datos de focos de calor. Esta función lee los archivos `.csv` de FIRMS, los convierte en un `GeoDataFrame` y filtra únicamente aquellos puntos que caen dentro de los límites del departamento de Cordillera. Estos puntos, etiquetados con `fire = 1`, constituyen la mues-

tra positiva, es decir, los casos confirmados de incendios que el modelo aprenderá a predecir.

### 3.2. Generación de Muestras Negativas (No-Incendios)

No existe un registro de “lugares donde no hubo un incendio”, por ello es necesario generar una muestra sintética de puntos negativos. El script emplea una estrategia de **muestreo de fondo inteligente** para crear puntos negativos realistas para el modelo.

1. **Balance de Clases:** Se decide generar un número de puntos negativos igual al de puntos positivos, creando un conjunto de datos perfectamente balanceado.
2. **Proceso de Generación Iterativo:** Se inicia un bucle que genera puntos aleatorios uno por uno hasta alcanzar el número deseado. Cada punto candidato debe pasar una serie de filtros rigurosos para ser aceptado:
  - **Filtro Espacial Primario:** El punto debe caer no solo dentro de los límites del departamento, sino también **dentro del polígono de zonas inflamables** previamente creado. Esto evita generar puntos “fáciles” en lugares como cuerpos de agua o centros urbanos densos donde un incendio forestal es imposible.
  - **Filtro Espacio-Temporal:** Para evitar que un punto negativo esté demasiado cerca de un incendio real (cosa que podría confundir al modelo), se aplica una restricción de proximidad. A cada punto candidato se le asigna una **fecha aleatoria** dentro del rango de estudio. Luego, se comprueba que no exista ningún incendio real en un radio de **15 kilómetros** y en una ventana temporal de **3 días** (antes y después) de esa fecha.

Solo si un punto candidato supera ambos filtros, se añade a la lista de puntos negativos (`puntos_negativos_list`) y se le asigna la etiqueta `fire = 0`.

### 3.3. Consolidación y Asignación de Variables Predictoras

1. **Combinación y Aleatorización:** Una vez generados los puntos negativos, se convierten en un `GeoDataFrame`. Este se concatena con el `GeoDataFrame` de puntos positivos. El conjunto de datos resultante (`sample`) se mezcla de forma aleatoria (`.sample(frac=1)`) para asegurar que los casos positivos y negativos

no estén agrupados, lo cual es crucial para el entrenamiento del modelo.

2. **Carga de Datos Climáticos:** Se llama a la función `load_climate_data` para obtener la serie temporal diaria de variables meteorológicas (temperatura, precipitación, viento y humedad).
3. **Enriquecimiento del Conjunto de Datos:** Finalmente, se invoca la función principal de ingeniería de características, `asignar_variables`. Esta función toma el `GeoDataFrame` combinado de puntos positivos y negativos y, para cada punto, extrae y asigna todas las variables predictoras:
  - **Topográficas:** Elevación, pendiente y orientación.
  - **Vegetación:** Cobertura del suelo y NDVI.
  - **Climáticas:** Temperatura, precipitación, humedad y viento correspondientes a su fecha.
  - **Antropogénicas:** Distancia a vías principales y ciudades.

El resultado de esta fase es el `full_dataset`, un `GeoDataFrame` completo y enriquecido que contiene tanto las observaciones (puntos de incendio y no-incendio) como todas las variables predictoras necesarias, listo para ser depurado y utilizado en la fase de modelado.

## 4. Depuración y Modelado Predictivo

Esta fase se divide en dos etapas críticas: la depuración de los datos para asegurar su calidad y el proceso de modelado, que incluye una estrategia de validación espacialmente consciente.

### 4.1. Depuración del Conjunto de Datos

1. **Detección de Nulos:** Se realiza un diagnóstico para identificar cuántos valores faltantes existen en cada columna del `full_dataset`.
2. **Estrategia de Eliminación:** se eliminan todas las filas (`.dropna()`) que contengan al menos un valor nulo. Aunque esto puede reducir el tamaño total de la muestra, garantiza que cada observación que ingresa al modelo sea completa, evitando errores y sesgos potenciales. Se realiza una comprobación para asegurar que, después de la depuración, quede un número suficiente de registros para un entrenamiento significativo.
3. **Preparación del DataFrame Final:**

- Se crea un nuevo `DataFrame` llamado `datos_modelo` que contiene exclusivamente las columnas que se utilizarán como variables predictoras y la variable objetivo (`fire`).
- Se convierte la columna `cobertura` al tipo de dato `category`, optimizando el uso de memoria y facilitando el tratamiento como variable categórica en `scikit-learn`.
- Finalmente, se guarda un resumen (`.info()`, `.head()`) del `DataFrame` final en un archivo de texto (`dataset_summary.txt`), asegurando la **reproducibilidad** del análisis.

## 4.2. Modelado con Validación Cruzada Espacial

1. **Autocorrelación Espacial:** Los datos geográficos tienen una propiedad inherente: los puntos cercanos en el espacio tienden a ser más similares entre sí. Si se utilizara una división aleatoria estándar (`train_test_split`), se correría el riesgo de que el conjunto de prueba contenga puntos muy similares a los del entrenamiento, inflando artificialmente la precisión del modelo.
2. **División Espacial por Bloques:** Se implementa una **validación cruzada espacial**. En lugar de una división aleatoria, el departamento se divide geográficamente: el 70% de los datos ubicados en el sur (según su latitud) se utilizan para **entrenar** el modelo, y el 30% restante, ubicado en el norte, se reserva para **evaluarlo**. Esta estrategia simula un escenario realista, donde el modelo debe generalizar a una región completamente nueva, proporcionando una medida más honesta del rendimiento.
3. **Pipeline de Preprocesamiento:** Se construye un Pipeline de `scikit-learn` para automatizar la preparación de los datos, asegurando consistencia entre entrenamiento y prueba.
  - **StandardScaler:** Se aplica a todas las variables numéricas (elevación, temperatura, etc.), normalizándolas con media 0 y desviación estándar 1.
  - **OneHotEncoder:** Se aplica a las variables categóricas (`orientacion_cat` y `cobertura`), transformándolas en columnas binarias para permitir el procesamiento por modelos numéricos.
4. **Arquitectura del Modelo:** `StackingClassifier`: **Stacking**.

- **Estimadores Base:** Se definen tres clasificadores distintos:

- `RandomForestClassifier`
- `LinearSVC` calibrado
- `KNeighborsClassifier`

Cada uno aprende de manera independiente a partir del conjunto de entrenamiento.

- **Meta-Modelo:** Se define una `LogisticRegression` como meta-modelo, que aprende a combinar las predicciones de los modelos base. Esto para ofrecer una mejor capacidad predictiva al aprovechar las fortalezas individuales de cada clasificador.

## 5. Entrenamiento y Evaluación:

- El Pipeline completo (preprocesamiento + stacking) se entrena con el conjunto de entrenamiento espacial (`X_train`, `y_train`).
- Luego se evalúa en el conjunto de prueba espacial (`X_test`, `y_test`), que representa una región no vista por el modelo.
- Se calculan métricas clave como:
  - **Accuracy:** Porcentaje total de aciertos.
  - **ROC AUC Score:** Capacidad del modelo para distinguir entre clases.
  - **Reporte de Clasificación:** Métricas detalladas de precisión, sensibilidad y F1 para cada clase (“Incendio” y “No Incendio”).

## 5. Modelado con Validación Espacial

### 5.1. La Autocorrelación Espacial

Tendencia de que los puntos cercanos en el espacio sean más similares entre sí que los puntos lejanos. Si se utilizara una división aleatoria convencional para separar los datos en entrenamiento y prueba, el modelo podría ser evaluado con puntos de prueba que son vecinos muy cercanos de los puntos de entrenamiento.

Este escenario conduce a una **evaluación inflada y engañosamente optimista**. El modelo podría obtener una alta puntuación no porque haya aprendido a generalizar patrones complejos, sino porque simplemente está memorizando características locales.

## 5.2. Validación por Bloques Espaciales

Se implementa una estrategia de **validación por bloques espaciales**. En lugar de una división aleatoria, el área de estudio se segmenta geográficamente utilizando la coordenada de latitud (*y*) de cada punto. Se establece una línea de corte en el percentil 70 de las latitudes, dividiendo el conjunto de datos en dos regiones geográficas distintas y sin solapamiento:

- **Conjunto de Entrenamiento (50% de los datos):** Corresponde a todos los puntos de datos ubicados en la porción **sur** del área de estudio. El modelo aprende los patrones únicamente a partir de la información de esta zona.
- **Conjunto de Prueba (50% de los datos):** Corresponde a todos los puntos ubicados en la porción **norte**. Esta región se mantiene completamente aislada durante el entrenamiento y se utiliza para evaluar la capacidad de generalización del modelo.

## 5.3. Proceso de Implementación

1. **División Geográfica:** Se extrae la coordenada de latitud (*y*) de la geometría de cada punto en el `datos_modelo` depurado.
2. **Cálculo del Punto de Corte:** Se calcula el valor de la latitud que corresponde al **percentil 50** (`split_point = datos_modelo['latitud'].quantile(0.5)`).
3. **Creación de los Conjuntos:** Se crean dos nuevos `DataFrames`, `train_df` y `test_df`, filtrando los datos originales según si su latitud es menor o mayor que el punto de corte.
4. **Entrenamiento:** El Pipeline completo, que encapsula tanto el preprocesamiento de datos (escalado y codificación) como el modelo de ensamble `StackingClassifier`, se entrena exclusivamente con los datos del bloque sur (`X_train`, `y_train`).
5. **Evaluación Robusta:** La evaluación final del rendimiento se realiza aplicando el modelo ya entrenado al conjunto de prueba del bloque norte (`X_test`, `y_test`).

Las métricas resultantes (Accuracy, ROC AUC, reporte de clasificación) reflejan, por tanto, un escenario mucho más exigente y realista: la capacidad del modelo para predecir el riesgo en un territorio geográfico para el cual no ha sido entrenado previamente. Esta metodología asegura que la evaluación del modelo no sea una mera prueba de memorización de patrones locales, sino una verdadera validación de su poder predictivo en nuevas áreas.

## 6. Análisis de Sanidad de la Muestra y la Partición Espacial

Para asegurar que los resultados del modelo son robustos y no producto de fallos metodológicos, se realiza una fase crítica de análisis de sanidad. Esta etapa busca validar dos hipótesis fundamentales sobre la calidad de los datos y la estrategia de evaluación.

### 6.1. Hipótesis 1: ¿Son las muestras negativas "demasiado fáciles"?

El proceso de generación de puntos de "no-incendio" es sintético y podría, si no se diseña con cuidado, crear casos que son trivialmente distinguibles de los incendios reales, inflando artificialmente el rendimiento del modelo. Para verificar la calidad de esta muestra negativa, se realizan dos análisis visuales:

- **Mapa de Distribución Espacial de la Muestra:** Se genera un mapa que muestra la ubicación de los puntos positivos (incendios, en rojo) y los negativos (no-incendios, en azul) sobre el área de estudio. El objetivo es confirmar visualmente que los puntos negativos se distribuyen en las mismas zonas geográficas generales que los positivos (es decir, en áreas inflamables) y no en lugares imposibles como centros urbanos o grandes cuerpos de agua. Una buena distribución, con puntos de ambos colores entremezclados, sugiere que la muestra negativa es espacialmente realista y relevante.
- **Comparación de Distribuciones por Clase:** Se utilizan diagramas de caja (`boxplot`) para comparar la distribución de las variables predictoras clave (como `elevacion`, `ndvi` o `dist_ciudades`) entre la clase "Incendio" y "No Incendio". Si las cajas muestran diferencias claras entre ambas clases, confirma que las variables tienen poder predictivo y que la estrategia de muestreo generó contraejemplos significativos, en lugar de puntos con características idénticas a los de los incendios.

### 6.2. Hipótesis 2: ¿Es la partición espacial representativa y justa?

La validación por bloques espaciales es una técnica poderosa, pero es fundamental entender cómo difieren las características entre la región de entrenamiento (sur) y la de prueba (norte). Una disparidad extrema podría significar que el modelo está siendo forzado a extrapolar en lugar de generalizar.

- **Mapa de Visualización de la Partición Espacial:** Se crea un mapa que colorea los puntos según pertenezcan al conjunto de entrenamiento o al de prueba, dibujando también la línea de corte latitudinal. Esto sirve para confirmar visualmente que la división geográfica se ha realizado correctamente, creando dos bloques espaciales distintos y sin solapamiento.
- **Comparación de Distribuciones entre Train y Test:** Se generan diagramas de caja para cada variable numérica, pero esta vez comparando la distribución entre el conjunto de **Train** y el de **Test**. Este es un chequeo de sanidad crucial para la estrategia de evaluación. Permite cuantificar visualmente cuán diferentes son las dos regiones geográficas en términos de sus características. Si bien se esperan algunas diferencias (lo que hace la evaluación más realista), este análisis asegura que las distribuciones no sean completamente disjuntas, lo que invalidaría la capacidad del modelo para generalizar.

En conjunto, estos análisis de sanidad proporcionan una fuerte evidencia de que el conjunto de datos generado es de alta calidad y que la estrategia de evaluación espacial es desafiante pero justa. Esto aumenta la confianza en que los resultados de rendimiento del modelo son fiables y reflejan su verdadera capacidad predictiva.

## 7. Resultados del Modelo

Una vez entrenado el modelo de Stacking sobre el bloque espacial sur y evaluado sobre el bloque norte, se procedió a analizar su rendimiento cuantitativo. Los resultados, guardados y visualizados desde el script, proporcionan una medida objetiva de la capacidad del modelo para predecir el riesgo de incendio en una zona geográficamente nueva.

### 7.1. Métricas de Rendimiento

Las métricas clave obtenidas en el conjunto de prueba espacial fueron las siguientes:

- **Accuracy (Exactitud):** Esta métrica indica el porcentaje total de predicciones correctas (tanto incendios como no-incendios) sobre el total de la muestra de prueba.
- **ROC AUC Score:** Un valor cercano a 1.0 indica una excelente capacidad del modelo para discriminar entre las clases positiva (incendio) y negativa (no-incendio).

El reporte de clasificación completo, que detalla la precisión y la sensibilidad (recall) para cada clase.

### 7.2. Visualización del Rendimiento

Para un análisis más intuitivo, se generó un gráfico combinado que incluye la matriz de confusión y la curva ROC.

La **matriz de confusión** revela que el modelo identificó correctamente XXX verdaderos positivos (incendios) y XXX verdaderos negativos, mientras que cometió XXX errores de tipo I (falsos positivos) y XXX errores de tipo II (falsos negativos). La **curva ROC**, por su parte, se eleva significativamente por encima de la línea de azar (línea discontinua), confirmando visualmente el buen poder discriminatorio del modelo indicado por el valor AUC.

## 8. Explicabilidad del Modelo (Análisis SHAP)

Para evitar que el modelo de Machine Learning actúe como una "caja negra", se implementó un análisis de explicabilidad utilizando la librería SHAP (SHapley Additive exPlanations). Este análisis se centró en el modelo RandomForest, uno de los componentes principales del ensamble, para entender qué variables predictoras tienen mayor impacto en las predicciones de riesgo de incendio.

### 8.1. Importancia de las Características

El gráfico de resumen de SHAP proporciona dos piezas clave de información: qué variables son las más importantes y cómo influyen en la predicción.



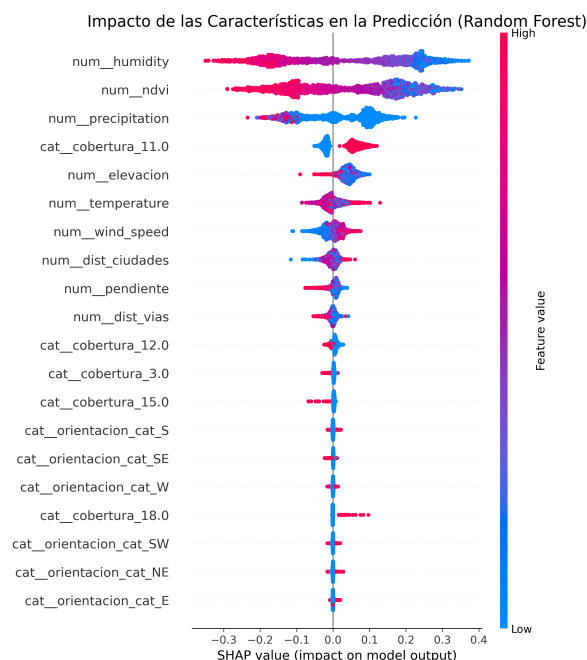


Figure 1: Gráfico de resumen SHAP (beeswarm plot). Cada punto es una predicción para una muestra. El color indica el valor de la característica (rojo=alto, azul=bajo) y su posición en el eje X indica su impacto en la predicción de incendio. Lado Positivo (Valor SHAP  $>$  0): La característica, con su valor específico para esa muestra, aumentó la probabilidad de incendio. Lado Negativo (Valor SHAP  $<$  0): La característica, con su valor específico para esa muestra, disminuyó la probabilidad de incendio.

El gráfico de barras de importancia media confirma el ranking de las variables, promediando el impacto absoluto de cada una. Este análisis no solo valida que el modelo está aprendiendo relaciones lógicas y esperadas (p.ej., más calor o cercanía a humanos aumenta el riesgo), sino que también proporciona información valiosa para los gestores de recursos, quienes pueden enfocar sus esfuerzos de prevención en las variables de mayor impacto.

## 9. Aplicación Práctica: Generación del Mapa de Riesgo

El objetivo final del pipeline es traducir el modelo predictivo en una herramienta visual y accionable. Para ello, se generó un mapa de riesgo de alta resolución para todo el departamento de Cordillera para una fecha específica

### 9.1. Metodología de Generación

El proceso para crear el mapa fue el siguiente:

#### 1. Creación de una Rejilla de Predicción:

Se generó una rejilla o *grid* de puntos uniforme que cubre toda la extensión del área de estudio.

**2. Asignación de Variables a la Rejilla:** A cada punto de la rejilla se le asignaron todas las variables predictoras (topográficas, de cobertura, NDVI, de proximidad y climáticas para la fecha de interés) utilizando la misma función `asignar_variables` usada para la muestra de entrenamiento.

**3. Predicción de Probabilidad:** El pipeline de modelo entrenado se aplicó a esta rejilla enriquecida para predecir la probabilidad de incendio (un valor entre 0 y 1) para cada punto.

**4. Interpolación y Visualización:** Los valores de probabilidad puntuales se interpolaron usando la función `griddata` para crear una superficie de riesgo continua. Esta superficie se visualizó como un mapa de calor (*heatmap*), donde los colores cálidos (rojo) indican alta probabilidad de incendio y los colores fríos (amarillo/blanco) indican baja probabilidad.

Este mapa es el producto más tangible del proyecto. Permite a los tomadores de decisiones identificar rápidamente los "puntos calientes" que requieren mayor vigilancia o la asignación prioritaria de recursos de prevención y combate, convirtiendo los datos y el modelo de Machine Learning en inteligencia accionable. Además, todas las predicciones puntuales se exportaron a un archivo `predicciones_riesgo.csv` para su uso en otras plataformas o análisis.

## 10. Algunos Conceptos

**Semillas aleatorias:** Un número inicial utilizado para generar una secuencia de números pseudoaleatorios. Fijar una semilla asegura que cualquier proceso que involucre aleatoriedad (como la división de datos o el entrenamiento de ciertos modelos) produzca exactamente el mismo resultado cada vez que se ejecuta el código, garantizando la **reproducibilidad** del experimento.

**Rejilla (Grid):** Un conjunto de puntos distribuidos uniformemente sobre un área geográfica. Se usa como base para aplicar el modelo entrenado a toda la región y así poder generar un mapa de riesgo continuo.

**Geoespaciales vectoriales:** Datos que representan características geográficas mediante puntos, líneas y polígonos. En este proyecto, los límites del departamento, las ciudades y las vías son datos vectoriales.

**GeoDataFrame:** La estructura de datos fundamental de la librería `geopandas`. Es como un `DataFrame` de `pandas`, pero con una columna especial ("geometry") que almacena objetos geométricos, permitiendo realizar operaciones espaciales.

**Raster:** Un formato de datos geoespaciales que representa el espacio como una matriz de celdas o píxeles, donde cada píxel tiene un valor (ej. elevación, temperatura, clase de cobertura). El DEM y los mapas de NDVI son datos raster.

**Arreglos N-dimensionales etiquetados:** La estructura de datos principal de `xarray`. Permite almacenar datos multidimensionales (como un raster con coordenadas, tiempo y bandas) usando etiquetas (nombres de dimensiones y coordenadas) en lugar de índices numéricos, lo que facilita selecciones y operaciones complejas.

**Normalizar:** El proceso de escalar las variables numéricas para que estén en un rango común. Se hace para que las variables con magnitudes muy grandes (como la elevación en metros) no dominen a las de magnitudes pequeñas (como el NDVI) durante el entrenamiento del modelo.

**StandardScaler:** Una técnica de normalización de `scikit-learn` que transforma los datos para que tengan una media de 0 y una desviación estándar de 1.

**OneHotEncoder:** Un preprocesador que convierte variables categóricas (como "Norte", "Sur", "Este") en columnas binarias (0s y 1s), un formato que los modelos de Machine Learning pueden entender.

**Pipeline:** Un objeto de `scikit-learn` que encadena múltiples pasos de procesamiento y modelado en un solo flujo de trabajo. Asegura que la misma secuencia de transformaciones se aplique de forma consistente a todos los datos.

**Ensamble:** Una técnica de Machine Learning que combina las predicciones de varios modelos individuales (estimadores base) para producir una predicción final más precisa y robusta que la de cualquier modelo por sí solo. El Stacking es un tipo de ensamble.

**Meta-modelo:** En un modelo de Stacking, es el clasificador final que toma las predicciones de los modelos base como entrada y aprende a combinarlas de la mejor manera para dar el resultado definitivo.

**Calibrador (CalibratedClassifierCV):** Un wrapper de `scikit-learn` que se utiliza para ajustar

las salidas de un modelo para que representen mejor una probabilidad real. Se usó sobre `LinearSVC` porque este no produce probabilidades de forma nativa.

**Boxplot (Diagrama de caja):** Una visualización estadística que representa la distribución de un conjunto de datos a través de sus cuartiles. Es útil para comparar rápidamente la mediana, la dispersión y los valores atípicos entre diferentes grupos.

**Heatmap (Mapa de calor):** Una visualización gráfica donde los valores de una matriz se representan como colores. Se usó para la matriz de confusión, donde los colores más intensos indican un mayor número de casos.

**Matriz de confusión:** Una tabla que resume el rendimiento de un modelo de clasificación. Compara las clases predichas con las clases reales y desglosa los resultados en verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

**Máscara de recorte (Clipping Mask):** Una forma o polígono utilizado para ocultar partes de otra capa visual. En el script, se usó el polígono del departamento para recortar el mapa de calor interpolado, asegurando que el riesgo solo se muestre dentro de los límites del área de estudio.

**Mapa de calor interpolado:** Una superficie continua de color creada a partir de valores en puntos discretos. La interpolación (como la cúbica) estima los valores en los espacios vacíos entre los puntos de la rejilla para crear una visualización suave y fácil de interpretar.

**Geometría poligonal:** Una forma vectorial cerrada que representa un área, definida por una secuencia de vértices conectados. El límite del departamento de Cordillera es una geometría poligonal.

**CRS (Coordinate Reference System):** Sistema de Coordenadas de Referencia. Es un estándar que define cómo las coordenadas de los datos geográficos se relacionan con lugares reales en la Tierra. Es crucial para asegurar que todas las capas de datos se alineen correctamente.

**Autocorrelación Espacial:** La propiedad inherente de los datos geográficos donde los valores de las mediciones en ubicaciones cercanas tienden a ser más similares que en ubicaciones lejanas. Es un factor clave a considerar en la validación de modelos espaciales.